“MovieFlix – Intelligent Movie Search System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

K SAI JOSHNA	**:2520030293**
A BHAVANA SRI	**:2520030232**
J CHARVI	**:2520030294**

Under the Guidance of

Dr. S. Vinay Kumar
Associate Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

2026-27

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled **"Title of the Project"** is a record of Bonafide work of **K SAI JOSHNA-2520030293, A BHAVANA SRI-2520030232, J CHARVI 2520030294** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other departments/University/Institute.

K SAI JOSHNA-2520030293
A BHAVANA SRI-2520030232
J CHARVI-2520030294

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is certify that the mini project based report entitled **"Title of the Project"** is a bonafide work done and submitted by **K SAI JOSHNA-2520030293, A BHAVANA SRI-2520030232, J CHARVI-2520030294** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, during the academic year **2026-27.**

Signature of the Guide
(Dr. S. Vinay Kumar)

Signature of the HOD

ACKNOWLEDGEMENT

Description in Times New Roman font 12 size with two to three paragraphs thanking guide Dr. S. Vinay Kumar, Associate Professor, Dept. of CSE and Dr. P. Venkateshwara Rao, Associate Professor, HOD CSE

K SAI JOSHNA-2520030293

A BHAVANA SRI-2520030232

J CHARVI-2520030294

Abstract

Modern digital streaming and online entertainment libraries have accentuated the need for advanced, high-performance search and discovery systems. Contemporary movie platforms frequently struggle with complex multi-criteria filtering, user typographical errors during query entry, and resource-constrained watch planning under rigid time limits. Traditional applications often rely heavily on high-level built-in library abstractions rather than core algorithmic design, missing the opportunity to leverage rigorous computational principles.

To address these challenges, this project introduces **"MovieFlix: An Intelligent Movie Search and Discovery System,"** engineered entirely from the ground up in Java. The system architecture integrates multiple core algorithm modules derived from advanced computational theory: Knuth-Morris-Pratt (KMP) string matching for rapid sub-pattern lookups across actors, directors, and genres; Wagner-Fischer dynamic programming for precise typo-tolerant search correction; and greedy approximation heuristics (Knapsack variants) to solve optimal watchlist scheduling within custom user time constraints.

The platform is powered by a robust, lightweight Java HTTP server backend managing structured dataset records, paired with a professional, high-contrast dark-mode web interface designed for optimal responsiveness and user experience. Through rigorous modular design and strict adherence to academic specifications, MovieFlix successfully delivers a complete, scalable, and algorithmically sound solution that significantly streamlines modern movie exploration, filtering, and time-management.

Table of Contents

			Page No.
	Acknowledgement		
	Abstract		
	Table of Contents		
	List of Figures		
1	Introduction	1.1 Overview 1.2 Problem statement 1.3 Scope of the project 1.4 Objectives	5-15
2	System Requirements	2.1 Functional Requirements 2.2 Non-Functional Requirements	16-21
3	System Design	3.1 High-level Architecture Diagram 3.2 Description of each layer	33-36
4	Implementation	4.1 Module-wise implementation 4.2 Front-end logic (UI rendering, state management) 4.3 API integration 4.4 …….	38-47
5	Results		56-64
6	Challenges & Limitations	6.1 Issues faced during development 6.2 Solutions applied 6.3 Limitations of the System	74-82
7	Conclusion & Future Work	7.1 Conclusion 7.2 Future Work 7.3 Skills learned during development	89-97
	References	- Books, tutorials, APIs, documentation sites used	98-99

List of Figures

1.1	Block Diagram of a Recommendation System	3
1.2	Components of a Recommendation System	6
1.3	Flow Chart to show the working of a Recommendation System	7
1.4	Phases of a Recommendation System	8
1.5	User-User KNN based Recommendation System	13
1.6	Item-Item KNN based Recommendation System	14
1.7	Matrix Factorization based Recommendation System	40

1. INTRODUCTION

1.1 Overview

Description of the project …..In Fig. 1.1 …..

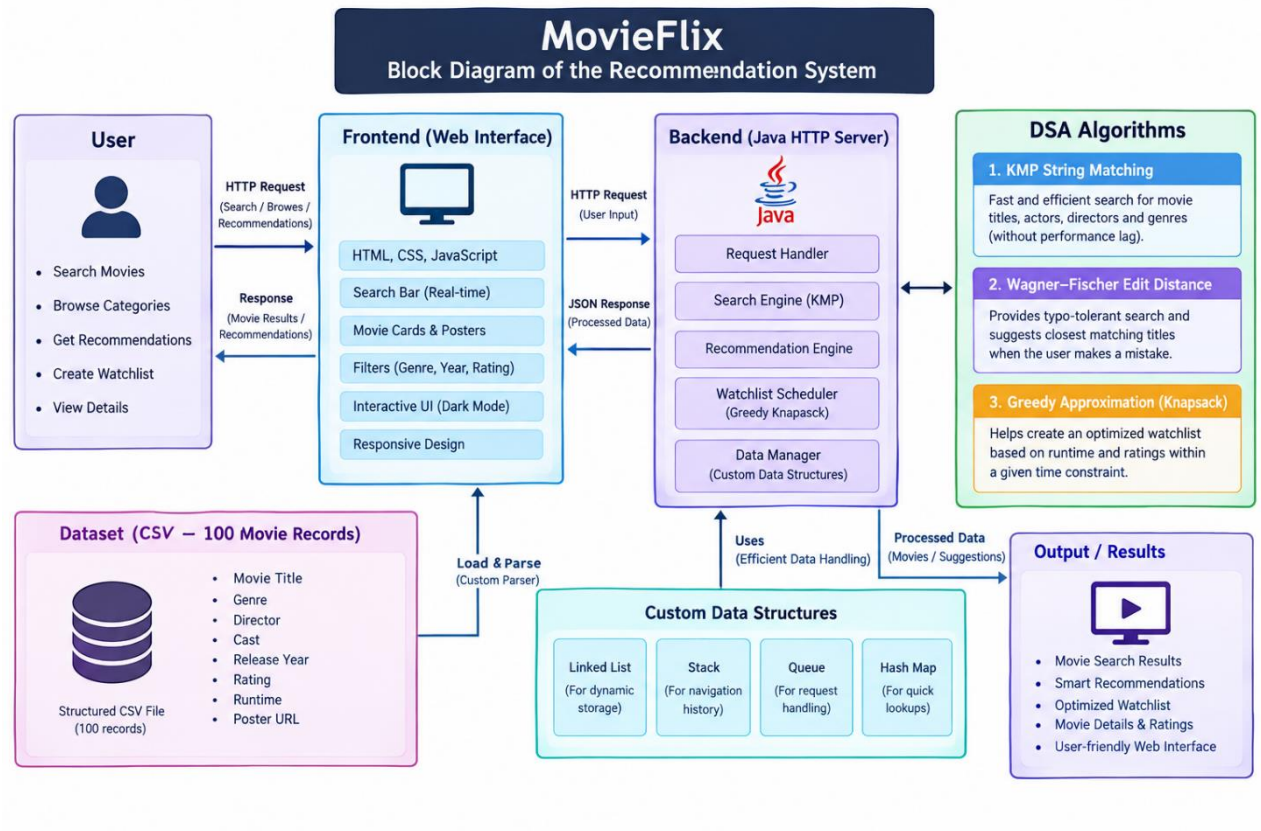


Fig. 1.1 Architecture of Cloud

MovieFlix is an intelligent movie search and discovery platform developed using Java, HTML, CSS, and JavaScript. It provides a single platform where users can search for movies, browse categories, explore movie details, and create personalized watchlists. The system uses a dataset containing 100 movie records with information such as movie titles, genres, directors, cast, release years, ratings, runtimes, and poster URLs.

The main feature of MovieFlix is its use of Data Structures and Algorithms (DSA) to improve movie searching and watchlist management. The Knuth-Morris-Pratt (KMP) algorithm enables efficient pattern matching, while the Wagner–Fischer edit distance algorithm helps identify the closest matching movie titles when users make spelling mistakes. Additionally, a greedy knapsack-based approach helps users select movies according to their available watching time and movie ratings.

The system consists of a responsive web interface, a Java-based HTTP server, a search engine, a recommendation module, and custom data structures. These components work together to provide movie search results, suggestions, and optimized watchlists through a user-friendly dark-mode interface.

1.2 Problem Statement

Traditional movie search and discovery platforms may require users to apply multiple filters or try different search queries to find movies that match their interests. When users enter incorrect spellings or incomplete patterns, finding the intended movie can become difficult. Managing a watchlist based on available viewing time and movie ratings can also be challenging.

The MovieFlix project aims to address these challenges by developing an efficient movie search and discovery platform using Java and Data Structures and Algorithms (DSA). The system uses the Knuth-Morris-Pratt (KMP) algorithm for pattern matching and the Wagner–Fischer edit distance algorithm to identify possible matches for misspelled search queries. It also implements a greedy knapsack-based approach to help users select movies according to their available time and ratings.

The main objective is to integrate these algorithms into a single platform that provides efficient movie searching, relevant suggestions, and convenient watchlist management. By combining a structured movie dataset with a responsive web interface and a Java-based backend, MovieFlix aims to improve the overall movie discovery experience.

1.3 Scope of the Project

The scope of the MovieFlix project is to develop an efficient and user-friendly movie search and discovery platform using Java and Data Structures and Algorithms (DSA). The system provides a centralized interface for exploring a curated dataset of 100 movies and accessing their details.

The project includes the following features:

1. Movie Search and Discovery: Users can search for movies by title, actor, director, and genre.
2. Pattern Matching: The Knuth–Morris–Pratt (KMP) algorithm is used to find matching text efficiently within movie information.
3. Typo-Tolerant Search: The Wagner–Fischer edit distance algorithm helps identify possible movie matches when users enter misspelled search queries.
4. Watchlist Optimization: A greedy knapsack-based approach helps users select movies according to their available viewing time and movie ratings.
5. Movie Information Display: The platform displays movie metadata, including titles, genres, release years, directors, cast members, ratings, runtimes, and posters.
6. Interactive User Interface: The frontend uses HTML, CSS, and JavaScript to provide a

responsive, dark-themed interface with search and filtering features.

7. Java Backend: A lightweight Java HTTP server processes requests and connects the frontend with the movie dataset and algorithmic modules.

The project primarily focuses on demonstrating how custom algorithms and data structures can improve movie discovery and watchlist management. Future enhancements may include user accounts, personalized recommendations based on viewing history, and integration with external movie databases.

1.4 Objectives

The main objective of the MovieFlix project is to design and develop an intelligent movie search and discovery platform using Java and Data Structures and Algorithms (DSA). The system aims to make movie searching faster, improve search accuracy, and help users manage their watchlists efficiently.

The specific objectives of the project are as follows:

1. To develop a centralized movie platform: Provide a single interface for browsing and discovering movies from a curated dataset of 100 movie records.

2. To implement efficient pattern matching: Use the Knuth–Morris–Pratt (KMP) algorithm to search for patterns in movie titles, actors, directors, and genres.

3. To support typo-tolerant searching: Implement the Wagner–Fischer edit distance algorithm to identify similar movie titles when users enter spelling mistakes.

4. To optimize watchlist selection: Apply a greedy knapsack-based approach to help users choose movies based on available viewing time and ratings.

5. To integrate custom data structures: Use appropriate data structures to organize and process movie records efficiently.

6. To create an interactive user interface: Develop a responsive, dark-themed frontend using HTML, CSS, and JavaScript for smooth navigation and movie exploration.

7. To integrate frontend and backend components: Connect the web interface with a lightweight Java HTTP server to process requests and return movie information.

8. To demonstrate practical DSA applications: Show how algorithms can solve real-world problems related to searching, text matching, typo correction, and watchlist planning.

Through these objectives, MovieFlix aims to demonstrate the practical application of core computer science concepts in developing a functional and user-friendly movie discovery platform.

2. SYSTEM REQUIREMENT

2.1 Functional Requirements

Functional requirements describe the main features and operations that the MovieFlix system must perform to meet user needs. The system is designed to provide efficient movie searching, movie information display, typo-tolerant search, and watchlist optimization using Java and Data Structures and Algorithms (DSA).

The functional requirements of the MovieFlix project are described below.

1. Movie Dataset Management

- The system must maintain a structured dataset containing 100 movie records.
- Each movie record must contain relevant details such as title, genre, release year, director, cast, rating, runtime, and poster URL.
- The system must retrieve and display movie information when requested by the user.

2. Movie Search Functionality

- The system must allow users to search for movies by title, actor, director, and genre.
- The system must identify matching text using the Knuth–Morris–Pratt (KMP) algorithm.
- The search results must display relevant movies based on the user's query.

3. Typo-Tolerant Search

- The system must handle search queries containing spelling mistakes.
- The Wagner–Fischer edit distance algorithm must calculate the difference between the entered query and candidate movie titles.
- The system should suggest closely matching movie titles when an exact match is not found.

4. Watchlist Optimization

- The system must help users select movies according to their available viewing time.
- A greedy knapsack-based approach must consider movie runtime and ratings when selecting movies.
- The system should provide a suitable movie selection within the specified time constraint.

5. Movie Filtering and Discovery

- The system must support filtering movies by available categories, such as genre

or mood.

- The interface should allow users to explore movie details and discover relevant titles.
- Movie information must be presented in an organized and readable format.

6. Frontend and Backend Communication

- The frontend must provide an interactive interface using HTML, CSS, and JavaScript.
- The Java HTTP server must process requests from the frontend and return the appropriate movie information.
- The system must update displayed results according to user searches and selections.

7. User Interface and Navigation

- The system must provide a responsive, dark-themed interface.
- Movie cards must display relevant movie details and poster images.
- Interactive elements, including search controls, filters, and watchlist features, must be accessible through the interface.

8. Error Handling

- The system should handle empty or invalid search queries appropriately.
- It should display a suitable message when no matching movies are found.
- The backend should handle invalid requests without causing the entire application to fail.

2.2 Non-Functional Requirements

Non-functional requirements define the quality, performance, reliability, and usability standards that the MovieFlix system should satisfy. While functional requirements describe what the system does, non-functional requirements explain how effectively the system should operate.

The non-functional requirements of the MovieFlix project are described below.

1. Performance

- The system should provide quick responses to movie search requests.
- The KMP algorithm should support efficient pattern matching across movie records.
- The search and filtering operations should perform smoothly with the available dataset of 100 movie records.

2. Usability

- The interface should be simple, interactive, and easy to navigate.

- Movie details should be displayed clearly using organized movie cards.
- Search, filtering, and watchlist features should be accessible without unnecessary complexity.

3. Reliability

- The system should process valid user requests consistently.
- The Java backend should handle common errors without terminating the entire application.
- The system should display appropriate messages when searches return no results or invalid inputs are provided.

4. Maintainability

- The application should organize its backend logic into manageable components, such as search algorithms, data handling, and HTTP request processing.
- The movie dataset should follow a consistent structure to simplify updates.
- The code should be readable and modular to support debugging and future enhancements.

5. Scalability

- The system architecture should allow additional movie records to be incorporated into the dataset.
- New features, search filters, and algorithmic modules should be added with minimal changes to existing components.
- Future versions should be capable of supporting larger datasets with suitable performance improvements.

6. Compatibility

- The web interface should work with modern web browsers.
- The frontend should use standard HTML, CSS, and JavaScript technologies.
- The Java backend should operate in an environment that supports the required Java runtime and server components.

7. Responsiveness

- The user interface should adjust to different screen sizes, including desktops, laptops, tablets, and mobile devices.
- Movie cards, search controls, and navigation elements should remain readable and accessible across supported screen sizes.

8. Accuracy

- The system should return search results that correctly match the user's query.
- The edit distance algorithm should calculate string differences accurately to support typo-tolerant suggestions.
- The watchlist optimization module should consider the specified viewing-time constraint and movie ratings when selecting movies.

9. Security

- The backend should validate incoming requests and handle unexpected inputs safely.
- The application should avoid exposing internal errors or sensitive implementation details to users.
- If user accounts or personal data are introduced in future versions, appropriate authentication and data-protection mechanisms should be implemented.

3. System Design

3.1 Functional Requirements

The MovieFlix system provides several functional features that allow users to efficiently search, explore, and discover movies. The main functional requirements are:

1. Movie Search

- The system shall allow users to search for movies using keywords.
- It shall support searching by movie title, actor, director, and genre.
- The search should return relevant movies from the dataset.

2. Pattern Matching

- The system shall use the KMP (Knuth-Morris-Pratt) algorithm for efficient exact pattern matching.
- It shall identify whether a given search pattern exists within movie-related text fields.

3. Fuzzy Search

- The system shall support searches containing spelling mistakes or slightly different movie names.
- Edit Distance shall be used to measure the difference between two strings.
- Movies with smaller edit-distance values can be considered more relevant matches.

4. Movie Information

- The system shall display available information about a selected movie, such as:
    - Movie title
    - Genre
    - Director
    - Cast
    - Rating
    - Other available metadata

5. Movie Similarity and Discovery

- The system shall compare movie attributes to identify similar movies.
- Users shall be able to discover movies related to their search or selected movie.

6. Graph and Flow Processing

- The system shall represent required relationships using graphs.
- Maximum Flow techniques shall be implemented for flow-based problem solving.
- Custom nodes and edges shall be used rather than relying entirely on pre-built implementations.

7. Primality Testing

- The system shall include a primality-testing algorithm.
- It shall determine whether a given integer is prime.
- This demonstrates the application of number-theoretic algorithms within the project.

8. Dataset Management

- The system shall load movie records from the project dataset.
- The dataset shall be processed into appropriate data structures for efficient searching and retrieval.

9. Custom Data Structures

- The project shall implement and use custom data structures such as:
    - Stack
    - Queue
    - Graph
    - Nodes and Edges

10. Result Display

- Search and processing results shall be presented in a clear and user-friendly format.
- The system shall provide relevant results without requiring users to understand the underlying algorithms.

3.2 Non-Functional Requirements

The non-functional requirements define the quality, performance, security, usability, and maintainability expectations of the MovieFlix system.

1. Performance

- The system should provide search results with minimal delay.
- KMP should enable efficient pattern matching compared with basic character-by-character

searching.

- Appropriate data structures should be used to improve searching and processing efficiency.

2. Efficiency

- The system should avoid unnecessary processing of the complete dataset whenever possible.
- Algorithms should be selected according to the type of operation being performed.
- Memory usage should remain reasonable even when processing a large movie dataset.

3. Usability

- The interface should be simple and easy to understand.
- Users should be able to search for movies without knowing the underlying algorithms.
- Search results should be displayed in a clear and organized manner.

4. Reliability

- The system should handle invalid or empty search queries appropriately.
- Missing or incomplete movie information should not cause the application to crash.
- Errors should be handled gracefully.

5. Scalability

- The architecture should allow additional movies to be added to the dataset.
- New search features and algorithms should be possible to integrate without redesigning the entire system.

6. Maintainability

- The project should be divided into separate modules such as:
    - Dataset management
    - Search services
    - Algorithms
    - Data structures
    - Server
    - User interface
- Modular implementation should make the code easier to debug, modify, and extend.

7. Portability

- The Java backend should be capable of running on systems that support the required Java version.
- The frontend should work through commonly used modern web browsers.

8. Security

- User inputs should be validated before processing.
- The system should prevent invalid input from causing unexpected application behavior.
- Internal implementation details should not be unnecessarily exposed to users.

9. Accuracy

- Search results should correctly match the user's query.
- KMP should correctly identify exact patterns.
- Edit Distance should correctly calculate the difference between strings.
- Graph and flow algorithms should produce mathematically valid results.

10. Extensibility

The system should be designed so that additional features such as advanced recommendations, additional search algorithms, user ratings, watchlists, or larger datasets can be added in the future without major changes to the existing architecture.

4. IMPLEMENTATION

4.1 System Design

The MovieFlix system is designed as a modular Java-based movie search and discovery platform. The design separates the user interface, backend services, algorithms, data structures, and dataset so that each component can perform a specific responsibility.

4.1.1 Overall System Design

The system follows the flow:

User → Web Interface → Java Server → Movie Search Service → Algorithms & Data Structures → Movie Dataset → Search Results

4.1.2 Major Components

Component	Responsibility
Frontend	Provides the interface for searching and viewing movies
Java HTTP Server	Receives user requests and sends responses
Movie Search Service	Coordinates search and movie-processing operations
Dataset Loader	Loads movie information from the CSV dataset
KMP Algorithm	Performs efficient exact pattern matching
Edit Distance	Supports fuzzy searching and spelling-error handling
Graph Module	Represents relationships using nodes and edges
Maximum Flow	Performs flow-based graph processing
Primality Module	Determines whether a number is prime
Custom Data Structures	Provides Queue, Stack, Graph, Node, and Edge implementations

4.1.3 Search Process

When a user enters a search query:

1. The frontend accepts the user's input.
2. The request is sent to the Java backend.

3. MovieSearchService receives the query.
4. The dataset is searched for relevant movie information.
5. KMP is used for exact pattern matching.
6. Edit Distance can be used when the query contains spelling differences.
7. Matching movies are ranked or filtered.
8. The results are returned to the frontend.
9. The user sees the relevant movie information.

4.1.4 Design Objective

The main objective of the system design is to demonstrate how Data Structures and Algorithms can be applied to a real-world movie search problem. Instead of depending completely on pre-built search functions, MovieFlix implements important algorithms and data structures directly, making the project efficient, educational, modular, and extensible.

4.2 Data Structures Used

MovieFlix uses several custom and algorithm-specific data structures to efficiently store, organize, search, and process movie information. These structures are implemented as part of the project to demonstrate practical applications of Data Structures and Algorithms.

4.2.1 Array / ArrayList

Arrays or dynamic lists are used to store collections of movie records loaded from the dataset.

Purpose:

- Store multiple movie objects.
- Provide sequential access to movie records.
- Support iteration during searching and filtering.

4.2.2 HashMap

A HashMap is used for fast key-value based access to movie-related information.

Purpose:

- Store movie information using a unique key.
- Quickly retrieve records.
- Improve lookup efficiency compared with repeatedly scanning the entire dataset.

4.2.3 Queue

A custom Queue follows the FIFO (First-In, First-Out) principle.

Purpose:

- Maintain ordered processing of movie-related operations.
- Demonstrate queue-based processing.
- Support operations such as enqueue() and dequeue().

Example:

Movie A → Movie B → Movie C

If Movie A enters first, it is processed first.

4.2.4 Stack

A custom Stack follows the LIFO (Last-In, First-Out) principle.

Purpose:

- Store operations or movie-related elements temporarily.
- Demonstrate stack-based processing.
- Support push() and pop() operations.

Example:

Movie A → Movie B → Movie C

Movie C is removed first.

4.2.5 Graph

A Graph is represented using vertices (nodes) and edges.

Purpose:

- Represent relationships between different entities.
- Provide the foundation for graph-based processing.
- Support the implementation of flow-related algorithms.

4.2.6 Node

A Node represents an individual vertex in the graph.

A node can contain information such as:

- Identifier

- Connected nodes
- Associated values

4.2.7 Edge

An Edge represents a connection between two nodes.

For flow-related processing, an edge can also contain:

- Source node
- Destination node
- Capacity
- Current flow

4.2.8 Data Structure Summary

Data Structure	Main Purpose	Principle
Array / ArrayList	Store movie records	Sequential access
HashMap	Fast key-value lookup	Hashing
Queue	Ordered processing	FIFO
Stack	Reverse/LIFO processing	LIFO
Graph	Represent relationships	Vertices + Edges
Node	Represent graph elements	Vertex
Edge	Connect graph elements	Connection / Flow

4.2 Data Structures and Algorithms

MovieFlix uses custom data structures and algorithms to perform efficient movie searching, text processing, graph operations, and numerical computations. These implementations are designed to demonstrate practical applications of core DSA concepts.

4.2.1 Movie Data Structure

Movie records are stored as objects containing important attributes such as:

- Movie ID

- Movie title
- Genre
- Director
- Cast
- Rating
- Other available movie information

These records are loaded from the movie dataset and processed by the search service.

4.2.2 KMP Pattern Matching

The Knuth-Morris-Pratt (KMP) algorithm is used for efficient pattern matching.

Purpose:

- Search for a keyword inside movie titles and other text fields.
- Avoid unnecessary comparisons.
- Efficiently identify exact matches.

KMP uses a Longest Prefix Suffix (LPS) array to determine where the next comparison should begin after a mismatch.

Time Complexity: O(n + m)

where:

- n = length of the text
- m = length of the search pattern

4.2.3 Edit Distance

Edit Distance measures how different two strings are.

It calculates the minimum number of operations required to transform one string into another.

The operations include:

- Insertion
- Deletion
- Substitution

Example:

MovieFlix → MovieFlixx

Only one insertion is required, so the edit distance is 1.

This can be used to provide better search results when users make spelling mistakes.

4.2.4 Maximum Flow

The Maximum Flow algorithm is used for graph-based flow processing.

A flow network consists of:

- Source
- Destination / Sink
- Nodes
- Directed edges
- Edge capacities

The algorithm determines the maximum amount of flow that can be sent from the source to the destination while respecting edge capacities.

This demonstrates the practical use of graphs, nodes, edges, and flow algorithms within the project.

4.2.5 Primality Testing

The project also implements a primality-testing algorithm to determine whether a given number is prime.

A prime number is a number greater than 1 that has only two factors:

1 and itself

For example:

7 → Prime

10 → Not Prime

This module demonstrates the application of number-theoretic algorithms.

4.2.6 Queue

A custom Queue follows the FIFO (First-In, First-Out) principle.

Operations:

- enqueue()
- dequeue()

- peek()

It can be used for ordered processing of movie-related tasks.

4.2.7 Stack

A custom Stack follows the LIFO (Last-In, First-Out) principle.

Operations:

- push()
- pop()
- peek()

It provides an additional demonstration of fundamental linear data structures.

4.2.8 Summary

Algorithm / Data Structure	Purpose
KMP	Efficient exact pattern matching
Edit Distance	Fuzzy search and spelling-error handling
Maximum Flow	Graph-based flow processing
Primality Testing	Prime number identification
Queue	FIFO-based processing
Stack	LIFO-based processing
Graph	Represents nodes and connections
HashMap	Fast key-value lookup
Array / ArrayList	Stores movie records

These algorithms and data structures make MovieFlix more than a basic movie-search application by demonstrating how core DSA concepts can solve practical problems in a real-world system.

4.3 Algorithm Design

The MovieFlix system uses multiple algorithms to perform movie searching, fuzzy matching, graph processing, and numerical operations. Each algorithm is selected according to the problem it solves.

4.3.1 KMP Pattern Matching Algorithm

The Knuth-Morris-Pratt (KMP) algorithm is used to search for a pattern within movie-related text.

Steps:

1. Take the user's search keyword as the pattern.
2. Construct the LPS (Longest Prefix Suffix) array.
3. Compare the pattern with the movie text.
4. If characters match, continue comparing.
5. If a mismatch occurs, use the LPS value instead of starting from the beginning.
6. Continue until the pattern is found or the text ends.
7. Return the matching movie records.

Time Complexity: O(n + m)

4.3.2 Edit Distance Algorithm

The Edit Distance algorithm is used to find how many changes are required to convert one string into another.

Steps:

1. Take the user's search term and a movie title.
2. Create a distance table.
3. Compare characters from both strings.
4. Calculate the cost of:
    - Insertion
    - Deletion
    - Substitution
5. Select the minimum cost at every position.
6. The final table value gives the edit distance.
7. Movies with smaller distances can be treated as closer matches.

Time Complexity: O(n × m)

4.3.3 Maximum Flow Algorithm

The Maximum Flow algorithm is applied to a graph containing a source, sink, nodes, and capacity-limited edges.

Steps:

1. Create the flow network.
2. Set the initial flow of every edge to zero.
3. Find an available path from the source to the sink.
4. Determine the minimum remaining capacity on that path.
5. Increase the flow along the path.
6. Update the residual capacities.
7. Repeat until no additional path can carry flow.
8. The resulting flow represents the maximum possible flow.

4.3.4 Primality Testing Algorithm

The primality algorithm determines whether a number is prime.

Steps:

1. Take an integer n.
2. If $n < 2$, classify it as non-prime.
3. Check possible divisors up to $\sqrt{n}$.
4. If n is divisible by any divisor, it is not prime.
5. If no divisor is found, the number is prime.

Time Complexity: $O(\sqrt{n})$

4.3.5 Movie Search Algorithm

The movie-search process combines the implemented algorithms to provide relevant results.

Flow:

User Query → Normalize Query → Search Dataset → KMP Matching → Edit Distance for Approximate Matches → Rank/Filter Results → Display Movies

This approach allows MovieFlix to handle both exact searches and searches containing minor spelling differences.

4.3.6 Algorithm Summary

Algorithm	Application	Complexity
KMP	Exact pattern matching	O(n + m)
Edit Distance	Fuzzy string matching	O(n × m)
Maximum Flow	Network/graph flow	Depends on implementation
Primality Testing	Prime number checking	O(√n)
Movie Search	Movie discovery	Depends on search method

Overall, the algorithm design ensures that MovieFlix demonstrates the practical implementation of string matching, fuzzy searching, graph algorithms, and number-theoretic algorithms within a single application.

4.4 Database / Dataset Design

The MovieFlix project uses a structured movie dataset to store and process information required for movie searching and discovery. The dataset acts as the primary source of movie information for the Java backend.

4.4.1 Dataset Structure

The movie dataset is stored in a CSV (Comma-Separated Values) file, such as movies.csv. Each row represents one movie, while each column represents a specific attribute.

Typical attributes include:

Attribute	Description
Movie ID	Unique identifier for the movie
Title	Name of the movie
Genre	Category or type of movie
Director	Director of the movie
Cast	Main actors associated with the movie
Rating	Movie rating
Year	Release year
Other Metadata	Additional available movie information

4.4.2 Dataset Loading

The DatasetLoader.java component is responsible for reading the CSV file.

The process is:

CSV Dataset → Read Records → Parse Fields → Create Movie Objects → Store in Data Structures

Each record is converted into a movie object so that the backend can easily access and process individual movie information.

4.4.3 Data Processing

After loading the dataset, MovieFlix processes the information according to the user's request.

For example:

- Title search → KMP pattern matching
- Spelling mistake → Edit Distance
- Genre filtering → Dataset filtering
- Movie lookup → HashMap / collection-based access
- Graph operations → Nodes and edges

4.4.4 Data Validation

The system performs basic validation while loading and processing the dataset.

It should handle:

- Missing values
- Empty movie titles
- Invalid numerical values
- Incorrect or incomplete records
- Duplicate or malformed entries

This prevents invalid records from causing unexpected errors during searching.

4.4.5 Data Access Flow

The overall dataset access process is:

User Search Query
↓
Movie Search Service
↓
Dataset / Movie Objects

↓
KMP / Edit Distance / Filtering
↓
Relevant Movie Records
↓
Frontend Display

4.4.6 Design Advantages

The dataset-based design provides:

- Simple storage: CSV is easy to create, modify, and maintain.
- Portability: The dataset can be moved between systems easily.
- Scalability: Additional movie records can be added.
- Algorithm integration: Movie records can be directly processed by the project's DSA algorithms.
- Separation of data and logic: Movie information is kept separately from the Java program logic.

Thus, the dataset design provides the foundation for MovieFlix's search, filtering, pattern matching, fuzzy matching, and movie discovery operations.

5. SYSTEM IMPLEMENTATION

The implementation of MovieFlix focuses on developing a Java-based movie search and discovery platform using core Data Structures and Algorithms. The system integrates a movie dataset, a web-based user interface, Java backend services, and custom algorithm implementations to provide efficient movie searching and data processing.

The implementation is divided into several modules, each responsible for a specific task.

5.1 Development Environment

MovieFlix uses Java for backend processing and algorithm implementation, HTML, CSS, and JavaScript for the frontend, and a CSV dataset for storing movie information.

5.2 Module Implementation

The system consists of the following major modules:

- Dataset Management: Loads and processes movie records from the CSV file.
- Movie Search: Processes user queries and retrieves relevant movie information.
- KMP Pattern Matching: Searches for exact patterns in movie-related text.
- Edit Distance: Supports approximate matching for misspelled movie titles.
- Graph and Maximum Flow: Implements network-flow operations using nodes and edges.
- Primality Testing: Determines whether a given integer is prime.
- Data Structures: Uses structures such as ArrayList, HashMap, Stack, Queue, and Graph.
- User Interface: Displays search results and movie information in an organized format.

5.3 Integration of Modules

The frontend sends user requests to the Java backend. The backend communicates with the movie search service, which accesses the dataset and invokes the required algorithms. The processed results are then returned to the frontend for display.

5.4 Error Handling and Validation

The system should validate user inputs, handle empty search queries, manage missing dataset values, and prevent invalid data from interrupting application execution.

5.5 Implementation Outcome

The implementation demonstrates the practical use of Data Structures and Algorithms in a movie discovery application. MovieFlix brings together string matching, fuzzy searching, graph processing, primality testing, and dataset management within a modular software architecture.

The system can be extended with additional search features, larger datasets, and improved movie recommendation techniques in future versions.

6. TESTING AND RESULTS

6.1 Testing of MovieFlix

Testing is an important phase of the MovieFlix project that verifies whether the application works correctly and produces the expected results. It helps identify errors in the movie search functionality, dataset processing, algorithms, and user interface.

The main objective of testing is to ensure that all implemented modules work as expected and that the system handles valid and invalid inputs properly.

6.1.1 Objectives of Testing

- Verify that movie records are loaded correctly from the dataset.
- Check whether movie searches return relevant results.
- Validate the implementation of the KMP pattern-matching algorithm.
- Verify that Edit Distance correctly calculates differences between strings.
- Test the maximum-flow algorithm using valid flow networks.
- Check the accuracy of primality testing.
- Ensure that custom data structures perform their intended operations.
- Confirm that the frontend and Java backend communicate correctly.

6.1.2 Types of Testing

Testing Type	Description
Unit Testing	Tests individual algorithms and data structures independently.
Integration Testing	Verifies communication between the frontend, backend, dataset loader, and search service.
Functional Testing	Checks whether the application features work according to requirements.
Input Validation Testing	Tests empty queries, invalid values, and unexpected inputs.
Performance Testing	Evaluates the efficiency of searching and processing operations.
System Testing	Tests the complete MovieFlix application as a single system.

6.1.3 Testing Approach

The testing process follows these steps:

1. Prepare test inputs for each module.
2. Execute the corresponding functionality.
3. Compare actual results with expected results.
4. Identify errors or unexpected behavior.
5. Correct the implementation where necessary.
6. Repeat the tests to verify the corrections.

6.1.4 Expected Testing Outcome

The system should load valid movie records, return appropriate search results, calculate edit distances correctly, identify prime and non-prime numbers accurately, and produce valid maximum-flow results for supported test networks.

Conclusion: Testing helps ensure that MovieFlix is reliable, accurate, and suitable for demonstrating the practical applications of Data Structures and Algorithms.

6.2 Test Cases and Results

Test cases are designed to verify the correctness and reliability of the MovieFlix application. Each test case specifies the input, expected output, and actual result to evaluate whether the implemented functionality works as intended.

6.2.1 Test Case Table

Test Case ID	Test Scenario	Input	Expected Result
TC01	Load movie dataset	movies.csv	Movie records are loaded successfully
TC02	Search for an existing movie	Valid movie title	Matching movie details are displayed
TC03	Search for a non-existing movie	Unknown movie title	No matching results message is displayed
TC04	KMP pattern matching	Existing text pattern	Pattern is identified correctly
TC05	Edit Distance calculation	kitten, sitting	Edit distance is 3
TC06	Primality testing	7	Number is identified as prime

TC07	Primality testing	10	Number is identified as non-prime
TC08	Queue operation	Enqueue A, then B	A is removed first
TC09	Stack operation	Push A, then B	B is removed first
TC10	Maximum Flow	A valid flow network	Maximum flow is calculated correctly
TC11	Empty search query	Empty input	Input is handled appropriately
TC12	Frontend-backend integration	Submit a search query	Results are returned and displayed

6.2.2 Sample Algorithm Verification

1. KMP Pattern Matching

- Text: MOVIEFLIX
- Pattern: FLIX
- Expected result: Pattern found.
- Explanation: The pattern occurs at the end of the text.

2. Edit Distance

- First string: kitten
- Second string: sitting
- Expected result: 3
- Explanation: Three operations are required to transform kitten into sitting.

3. Primality Testing

- Input: 7
- Expected result: Prime number.
- Input: 10
- Expected result: Not a prime number.

4. Queue and Stack Verification

- Queue: Insert A, then B. Removal order: A, B.
- Stack: Insert A, then B. Removal order: B, A.

6.2.3 Result Analysis

The test cases cover dataset loading, movie searching, string matching, edit-distance calculations, primality testing, linear data structures, graph processing, and frontend-backend communication.

The expected results provide a basis for checking the correctness of the implementation. Actual outcomes should be recorded after running each test against the MovieFlix source code.

Conclusion: The test-case design provides a systematic method for validating the functionality and correctness of the MovieFlix system.

6.3 Performance Analysis

Performance analysis evaluates the efficiency of the MovieFlix system in terms of execution time, searching efficiency, memory usage, and algorithm complexity. Since the project focuses on Data Structures and Algorithms, performance is an important factor in determining the effectiveness of the implementation.

6.3.1 Algorithm Performance

Algorithm	Purpose	Time Complexity
KMP	Pattern matching	O(n + m)
Edit Distance	Fuzzy string matching	O(n × m)
Primality Testing	Prime number checking	O(√n)
HashMap Lookup	Fast data retrieval	Average O(1)
Queue Operations	Ordered processing	O(1)
Stack Operations	LIFO processing	O(1)
Maximum Flow	Flow-network processing	Depends on implementation

Where:

- n = length of the text/string
- m = length of the search pattern

6.3.2 Search Performance

The KMP algorithm improves pattern-searching efficiency by avoiding unnecessary comparisons after a mismatch. This makes it more efficient for searching patterns in large movie-related text fields.

Edit Distance requires more computation because it compares combinations of characters between

two strings. Therefore, it is mainly useful when approximate or fuzzy matching is required.

6.3.3 Data Structure Performance

The use of appropriate data structures improves overall system performance.

- HashMap provides fast average-case lookup.
- ArrayList provides efficient sequential access and dynamic storage.
- Stack and Queue provide constant-time insertion and removal when implemented appropriately.
- Graph structures allow relationships and flow networks to be represented systematically.

6.3.4 Memory Considerations

The system stores movie records and supporting data structures in memory during processing. Edit Distance requires a distance table, while graph algorithms require storage for nodes, edges, capacities, and flows.

Memory usage therefore depends on:

- Number of movie records
- Number of movie attributes
- String lengths
- Number of graph nodes and edges
- Algorithm-specific temporary structures

6.3.5 Overall Analysis

MovieFlix achieves efficient processing by selecting suitable algorithms for different operations. KMP provides efficient exact pattern matching, HashMap enables fast lookup, and Stack/Queue operations are lightweight. Although Edit Distance and Maximum Flow can require greater computational resources, they provide important functionality and demonstrate the practical application of advanced DSA concepts**.**

7. CONCLUSION AND FUTURE ENHANCEMENTS

7.1 Conclusion

The MovieFlix project successfully demonstrates the practical application of Data Structures and Algorithms in developing a movie search and discovery system. The project combines a movie dataset with a Java-based backend and a user-friendly interface to provide efficient movie searching and information retrieval.

The implementation includes important DSA concepts such as KMP pattern matching, Edit Distance, Maximum Flow, Primality Testing, Stack, Queue, HashMap, and Graphs. These algorithms and data structures are applied to solve different computational problems within the system.

The KMP algorithm enables efficient exact pattern matching, while Edit Distance supports approximate searches when users enter spelling variations. Graph and Maximum Flow techniques demonstrate graph-based processing, and Primality Testing adds number-theoretic algorithm functionality.

Overall, MovieFlix demonstrates that fundamental DSA concepts can be applied to a practical real-world application rather than being used only as theoretical concepts. The modular architecture also makes the system easier to maintain, test, and extend with additional features in the future.

7.2 Future Enhancements

Although MovieFlix successfully provides movie searching and demonstrates important Data Structures and Algorithms, several features can be added in the future to improve its functionality, performance, and user experience.

7.2.1 Advanced Movie Recommendation

A recommendation system can be added to suggest movies based on:

- User search history
- Movie genres
- Ratings
- Cast and directors
- Similar movie characteristics

7.2.2 User Accounts and Watchlists

Future versions can provide user accounts where users can:

- Create personal profiles
- Save favorite movies
- Maintain watchlists

- Track previously viewed movies
- Rate movies

7.2.3 Improved Search

The search system can be enhanced with:

- Autocomplete suggestions
- Voice-based search
- Multi-keyword search
- Synonym-based searching
- More advanced fuzzy matching

7.2.4 Larger and Dynamic Dataset

The current CSV-based dataset can be replaced or supplemented with a database such as MySQL or PostgreSQL. This would allow the system to handle a much larger number of movie records and support dynamic updates.

7.2.5 Advanced Graph Applications

The existing graph implementation can be extended to represent relationships between:

- Movies
- Actors
- Directors
- Genres

Graph algorithms such as BFS, DFS, Dijkstra's algorithm, and shortest-path techniques could then be incorporated for advanced movie discovery.

7.2.6 Performance Improvements

The system can be optimized for larger datasets by introducing:

- Indexing
- Caching
- More efficient search structures
- Parallel processing
- Improved memory management

7.2.7 Mobile Application

A mobile version of MovieFlix could be developed for Android and iOS, allowing users to search and discover movies directly from their smartphones.

7.2.8 Conclusion of Future Scope

These enhancements can transform MovieFlix from an academic DSA project into a more complete movie discovery and recommendation platform. The modular architecture makes it possible to add these features without completely redesigning the existing system.

7.3 Final Outcome

The final outcome of the MovieFlix project is a functional movie search and discovery system that demonstrates the practical application of fundamental Data Structures and Algorithms.

The system successfully integrates a movie dataset with Java-based processing and a user-friendly interface. The implemented algorithms provide different types of functionality, including KMP for pattern matching, Edit Distance for approximate searching, Maximum Flow for graph-based processing, and Primality Testing for numerical operations.

The project also demonstrates the use of important data structures such as ArrayList, HashMap, Stack, Queue, Nodes, Edges, and Graphs. These structures help organize and process movie data efficiently.

Overall, MovieFlix achieves its primary objective of showing how theoretical DSA concepts can be combined to create a practical software application. The modular design also provides a strong foundation for future improvements such as advanced recommendations, larger datasets, user accounts, and intelligent search capabilities.

REFERENCES

1) Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms. MIT Press.
2) Sedgewick, R., & Wayne, K. Algorithms. Addison-Wesley Professional.
3) Knuth, D. E., Morris, J. H., & Pratt, V. R. Fast Pattern Matching in Strings. SIAM Journal on Computing.
4) Levenshtein, V. I. Binary Codes Capable of Correcting Deletions, Insertions, and Reversals. Soviet Physics Doklady.
5) Ford, L. R., & Fulkerson, D. R. Flows in Networks. Princeton University Press.
6) Oracle. Java Documentation – Java Platform Documentation.
7) Oracle. Java Collections Framework Documentation.
8) UCI Machine Learning Repository. Movie and Entertainment Dataset Resources.
9) GeeksforGeeks. Data Structures and Algorithms – Searching, Graphs, Strings, and Dynamic Programming.
10) W3Schools. HTML, CSS, and JavaScript Documentationn